



1

Preparing for Full-Stack Development

In this chapter, I am first going to give a brief overview of the contents of the book and explain why the skills taught in this book are important in a modern development environment. Then, we will jump into action and set up a project that will be used as a basis for the development of our full-stack projects. At the end of this chapter, you will have an **integrated development environment (IDE)** and project set up and ready for full-stack development and will understand which tools can be used for setting up such projects.

In this chapter, we are going to cover the following main topics:

- Motivation to become a full-stack developer
- What is new in the third edition?
- Getting the most out of this book
- Setting up the development environment

Technical requirements

This chapter will guide you through setting up all the necessary technologies needed for developing full-stack web applications throughout this book. Before we start, please install the following, if you do not already have them installed:

- Node.js v20.10.0
- Git v2.43.0
- Visual Studio Code v1.84.2

Those versions are the ones used in the book. While installing a newer version should not be an issue, please note that certain steps might work differently on a newer version. If you are having an issue with the code and steps provided in this book, please try using the mentioned versions.

You can find the code for this chapter on GitHub:

<https://github.com/PacktPublishing/Modern-Full-Stack-React-Projects/tree/main/ch1>.

The CiA video for this chapter can be found at:

<https://youtu.be/dyf3nECvKAE>.

IMPORTANT

If you cloned the full repository for the book, Husky may not find the `.git` directory when running `npm install`. In that case, just run `git init` in the root of the corresponding chapter folder.

Motivation to become a full-stack developer

Understanding full-stack development is becoming increasingly important, as companies seek to increase the cooperation – and reduce the gap – between the frontend and the backend. The frontend is becoming more deeply integrated with the backend, using technologies such as server-side rendering. Throughout this book, we are going to learn about the development, integration, testing, and deployment of full-stack projects.

What is new in this release of Full-Stack React Projects?

Unlike previous releases of Full-Stack React Projects, this new release focuses more on the integration of the frontend with the backend than the previous two editions, and thus intentionally does not focus so much on creating a **user interface (UI)** or using UI libraries, such as **Material UI (MUI)**, on the frontend. This edition gives the essential knowledge for integrating and deploying full-stack web applications. The deployment of apps was missing completely from previous editions, and testing was only briefly introduced. This edition focuses more on these essential parts of full-stack development such that, after reading this book, you will be able to develop, integrate, test, and deploy a full-stack web application.

Getting the most out of this book

To keep the book short and to the point, we are going to use specific technologies and tools. The concepts, however, apply to other technologies as well. We will attempt to briefly introduce alternatives so that if something is not a good fit for your project, you can pick and choose different tools. I recommend first trying out the technologies introduced in this book to be able to follow the instructions, but do not hesitate to try out the alternatives on your own later.

It is highly recommended that you write the code on your own. Do not simply run the code examples that are provided. It is important to write the code yourself in order to learn and understand it properly. However, if you run into any issues, you can always refer to the code examples.

With that said, let's start with setting up our development environment in the next section.

Setting up the development environment

In this book, we are going to use **Visual Studio Code (VS Code)** as our code editor. Feel free to use whichever editor you prefer, but keep in mind that the extensions used and settings configured may be slightly different in the editor of your choice.

Let's now install VS Code and useful extensions, and then continue setting up all the tools needed for our development environment.

Installing VS Code and extensions

Before we can get started developing and setting up the other tools, we need to set up our code editor by following these steps:

1. Download VS Code for your operating system from the official website (at the time of writing, the URL is <https://code.visualstudio.com/>). We are going to use version 1.84.2 in this book.
2. After downloading and installing the application, open it, and you should see the following window:

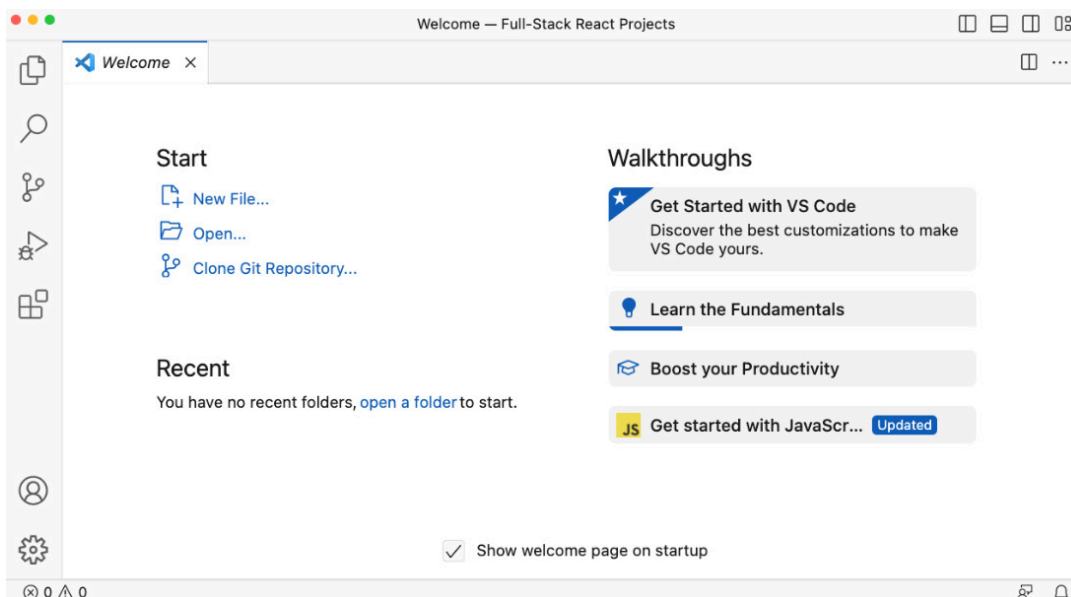


Figure 1.1 – A fresh installation of VS Code (on macOS)

3. To make things easier later, we are going to install some extensions, so click on the **Extensions** icon, which is the fifth icon from the top on the left in the screenshot. A sidebar should open, where you will see **Search Extensions in Marketplace** at the top. Enter an extension name here and click on **Install** to install it. Let's start by installing the **Docker** extension:

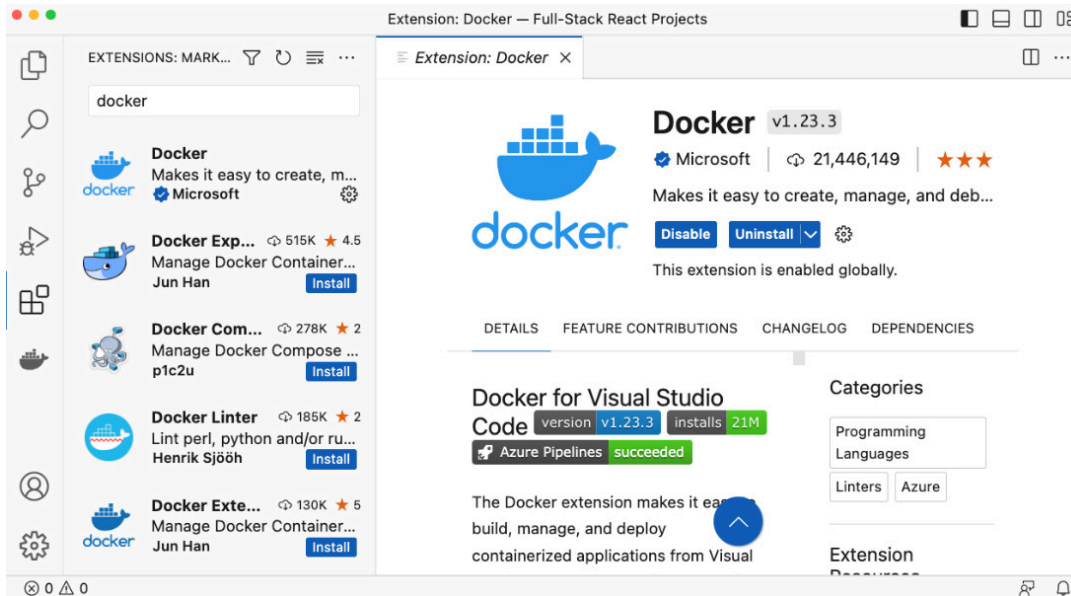


Figure 1.2 – Installing the Docker extension in VS Code

4. Install the following extensions:
 1. Docker (by Microsoft)
 2. ESLint (by Microsoft)
 3. Prettier – Code formatter (by Prettier)
 4. MongoDB for VS Code (by MongoDB)Support for JavaScript and Node.js already comes built-in with VS Code.
5. Create a folder for the projects made in this book (for example, you can call it **Full-Stack-React-Projects**). Inside this folder, create a new folder called **ch1**.
6. Go to the **Files** tab (first icon from top) and click the **Open Folder** button to open the empty **ch1** folder.

7. If you get a dialog asking **Do you trust the authors of the files in this folder?**, check **Trust the authors of all files in the parent folder 'Full-Stack-React-Projects'** and then click on the **Yes, I trust the authors** button.

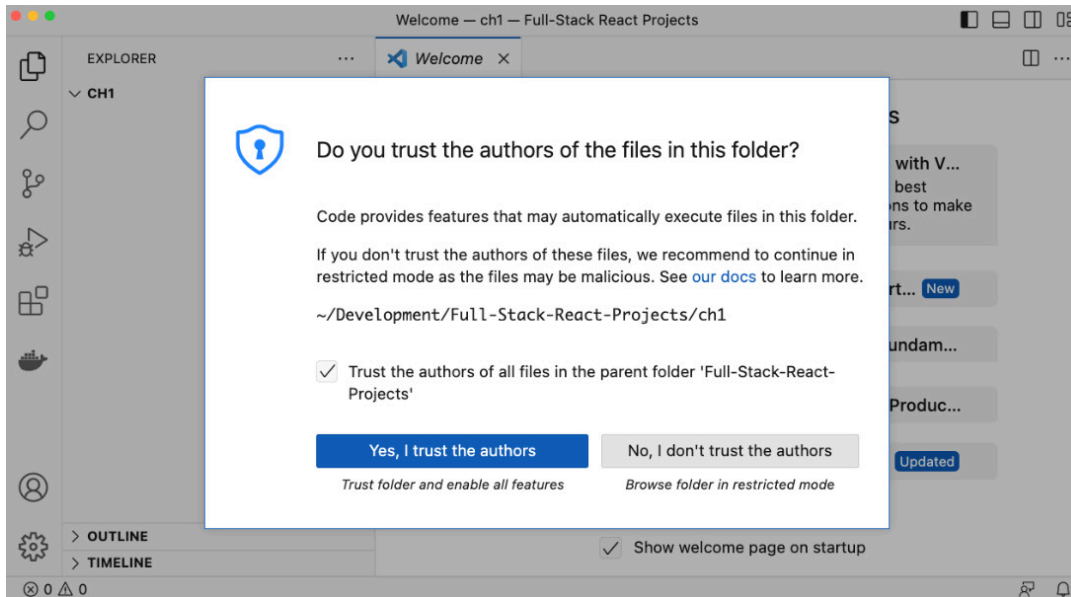


Figure 1.3 – Allowing VS Code to execute files in our project folder

TIP

*You can safely ignore this warning in your own projects, as you can be sure that those do not contain malicious code. When opening folders from untrusted sources, you can press **No, I don't trust the authors**, and still browse the code. However, when doing so, some features of VS Code will be disabled.*

We have now successfully set up VS Code and are ready to start setting up our project! If you have cloned the folder from the GitHub code examples provided, a notification telling you that a Git repository was found will also pop up. You can simply close this one, as we only want to open the **ch1** folder.

Now that VS Code is ready, let's continue by setting up a new project with Vite.

Setting up a project with Vite

For this book, we are going to use **Vite** to set up our project, as it is the most popular and liked according to *The State of JS 2022* survey (<https://2022.stateofjs.com/>). Vite also makes it easy to set up a modern frontend project, while still making it possible to extend the configuration later if needed. Follow these steps to set up your project with Vite:

1. In the VS Code menu bar, go to **Terminal** | **New Terminal** to open a new Terminal.
2. Inside the Terminal, run the following command:

```
$ npm create vite@5.0.0 .
```

Make sure there is a period at the end of the command to create the project in the current folder instead of creating a new folder.

NOTE

To keep the instructions in this book working even when new versions are released, we pin all packages to a fixed version. Please follow the instructions with the given versions. After finishing this book, when starting new projects on your own, you should always try using the latest versions but keep in mind that changes might be needed to get them working. Consult the documentation of the respective packages and follow the migration path from the book version to the latest version.

3. When asked if **create-vite** should be installed, simply type **y** and press the *Return/Enter* key to proceed.
4. When asked about the framework, use the arrow keys to select **React** and press *Return*. If you are being asked for a project name, press **Ctrl + C** to cancel, then run the command again, making sure there is a period at the end to select the current folder.
5. When asked about the variant, select **JavaScript**.

6. Now, our project is set up and we can run **npm install** to install the dependencies.
7. Afterward, run **npm run dev** to start the dev server, as shown in the following screenshot:

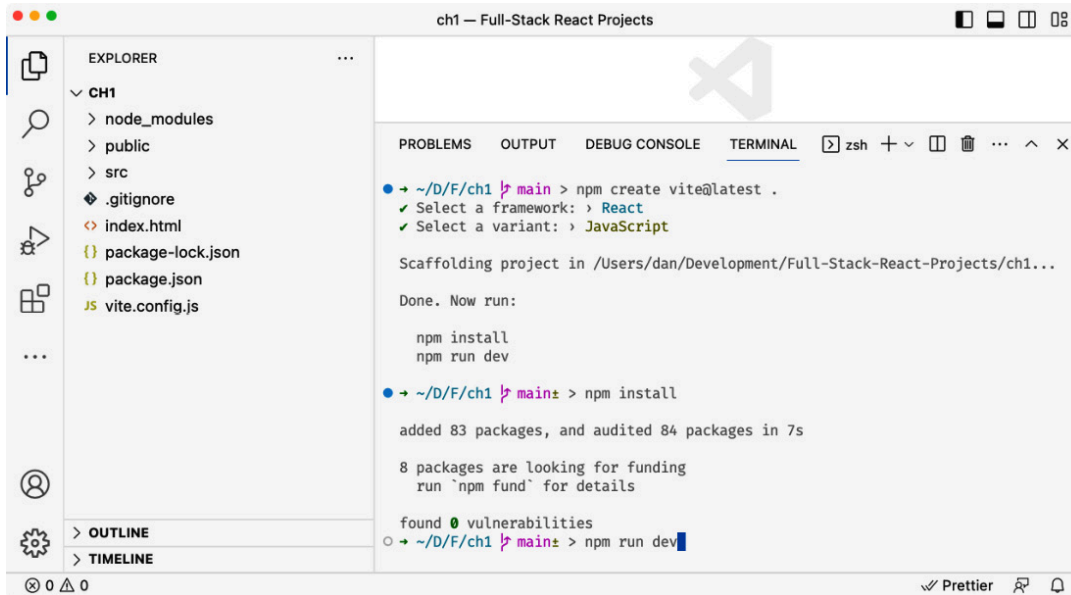


Figure 1.4 – The Terminal after setting up a project with Vite and before starting the dev server

NOTE

*For simplicity in setting up, we just used **npm** directly. If you prefer **yarn** or **pnpm**, you can instead run **yarn create vite** or **pnpm create vite**, respectively.*

8. In the Terminal, you will see a URL telling you where your app is running. You can either hold **Ctrl** (**Cmd** on macOS) and click on the link to open it in your browser, or manually enter the URL in a browser.
9. To test whether your app is interactive, click the button with the text **count is 0**, and it should increase the count every time it is pressed.

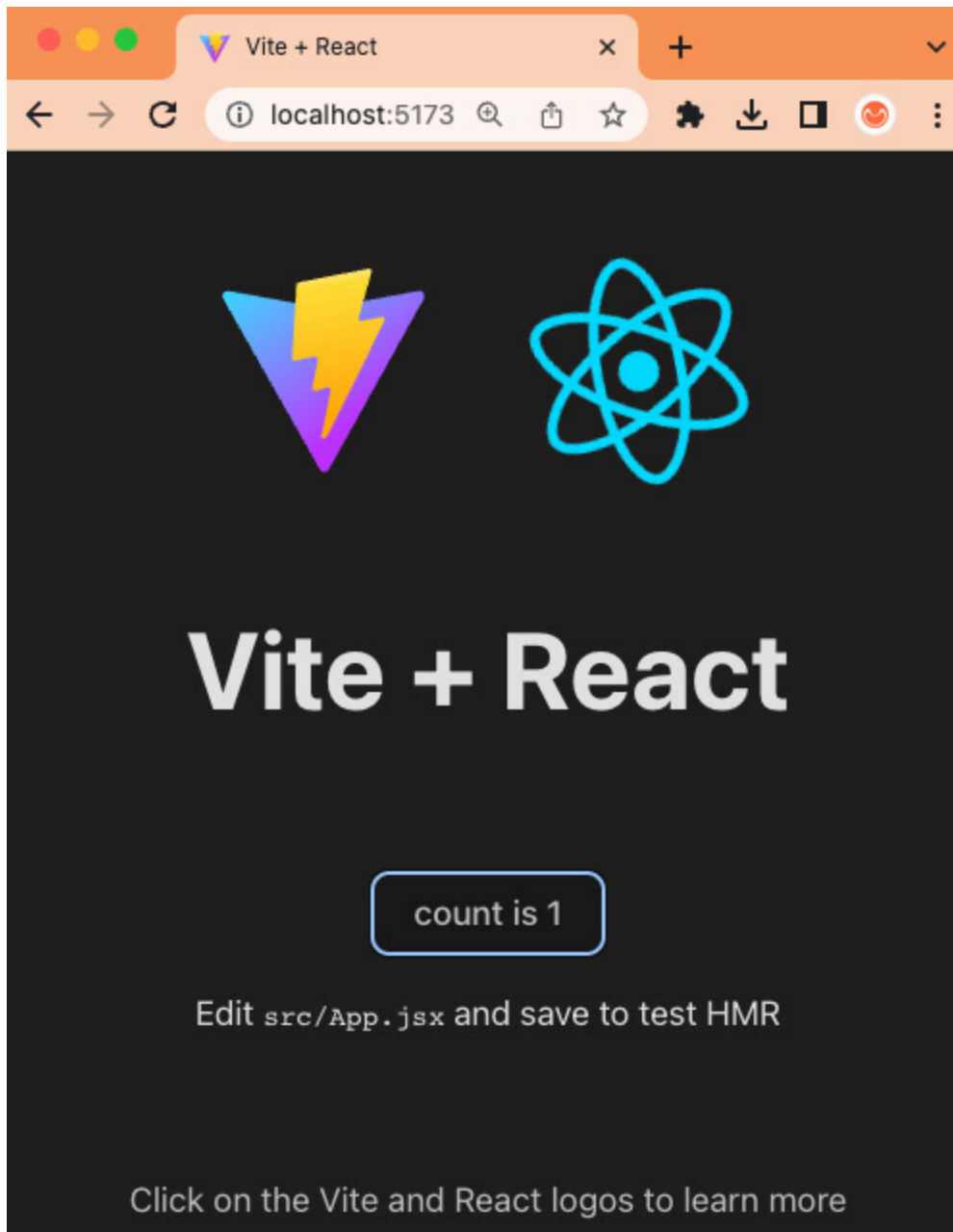


Figure 1.5 – Our first React app running with Vite

Alternatives to Vite

Alternatives to Vite are bundlers, such as webpack, Rollup, and Parcel. These are highly configurable but often do not offer a great experience for dev servers. They first must bundle all our code together before serving it to the browser. Instead, Vite natively supports the **ECMAScript module (ESM)** standard. Furthermore, Vite requires very little configuration to get started. A downside of Vite is that it can be hard to configure certain more complex scenarios with it. An upcoming bundler that is promising is Turbopack; however, it is still very

new at the time of writing. For full-stack development with server-side rendering, we will later get to know Next.js, which is a React framework that also provides a dev server out of the box.

Now that our boilerplate project is up and running, let's spend some time setting up tools that will enforce best practices and a consistent code style.

Setting up ESLint and Prettier to enforce best practices and code style

Now that our React app is set up, we are going to set up **ESLint** to enforce coding best practices with JavaScript and React. We are also going to set up **Prettier** to enforce a code style and automatically format our code.

Installing the necessary dependencies

First, we are going to install all the necessary dependencies:

1. In the Terminal, click on the **Split Terminal** icon at the top right of the **Terminal** pane to create a new **Terminal** pane. This will keep our app running while we run other commands.
2. Click on this newly opened pane to focus it. Then, enter the following command to install ESLint, Prettier, and the relevant plugins:

```
$ npm install --save-dev prettier@3.1.0 \
  eslint@8.54.0 \
  eslint-plugin-react@7.33.2 \
  eslint-config-prettier@9.0.0 \
  eslint-plugin-jsx-a11y@6.8.0
```

The packages installed are the following:

1. **prettier**: Formats our code automatically according to a defined code style
2. **eslint**: Analyzes our code and enforces best practices

3. **eslint-config-react**: Enables rules in ESLint relevant to React projects
4. **eslint-config-prettier**: Disables rules relating to code style in ESLint so that Prettier can handle them instead
5. **eslint-plugin-jsx-a11y**: Allows ESLint to check for accessibility (**a11y**) issues in our JSX code

NOTE

*The **--save-dev** flag in **npm** saves those dependencies as **dev** dependencies, which means that they will only be installed for development. They will not be installed and included in a deployed app. This is important in order to keep the size of our containers as small as possible later.*

After the dependencies are installed, we need to configure Prettier and ESLint. We will start with configuring Prettier.

Configuring Prettier

Prettier will format the code for us and replace the default code formatter for JavaScript in VS Code. It will allow us to spend more time writing code, automatically formatting it for us properly when we save the file. Follow these steps to configure Prettier:

1. Right-click below the files list in the left sidebar of VS Code (if it is not opened, click the **Files** icon) and press **New file...** to create a new file. Call it **.prettierrc.json** (do not forget the period at the beginning of the file name!).
2. The newly created file should open automatically, so we can start writing the following configuration into it. We first create a new object and set the **trailingComma** option to **all** to make sure objects and arrays that span over multiple lines always have a comma at the end, even for the last element. This reduces the number of touched lines when committing a change via Git:

```
{  
  "trailingComma": "all",
```

3. Then, we set the **tabWidth** option to **2** spaces:

```
"tabWidth": 2,
```

4. Set the **printWidth** to **80** characters per line to avoid long lines in our code:

```
"printWidth": 80,
```

5. Set the **semi** option to **false** to avoid semicolons where not necessary:

```
"semi": false,
```

6. Finally, we enforce the use of single quotes instead of double quotes:

```
  "jsxSingleQuote": true,  
  "singleQuote": true  
}
```

NOTE

These settings for Prettier are just an example of a coding style convention. Of course, you are free to adjust these to your own preferences. There are many more options, all of which can be found in the Prettier docs (<https://prettier.io/docs/en/options.html>).

Configuring the Prettier extension

Now that we have a configuration file for Prettier, we need to make sure the VS Code extension is properly configured to format the code for us:

1. Open the VS Code settings by going to **File | Preferences... | Settings** on Windows/Linux, or **Code | Settings... | Settings** on macOS.
2. In the newly opened settings editor, click on the **Workspace** tab. This ensures that we save all our settings in a **.vscode/settings.json** file in our project folder. When other developers open our project, they will automatically be using those settings as well.
3. Search for **editor format on save** and check the checkbox to enable formatting code on save.
4. Search for **editor default formatter** and select **Prettier - Code formatter** from the list.
5. To verify that Prettier works, open the **.prettierrc.json** file, add some extra spaces to the beginning of a line, and save the file. You should notice that Prettier reformatted the code to adhere to the defined code style. It will reduce the number of spaces for indentation to two.

Now that Prettier is set up properly, we do not need to worry about formatting our code manually anymore. Feel free to just type in code as you go and save the file to get it formatted for you!

Creating a Prettier ignore file

To improve performance and avoid running Prettier on files that should not be automatically formatted, we can ignore certain files and folders by creating a Prettier ignore file. Follow these steps:

1. Create a new file called **.prettierignore** in the root of our project, similar to how we created the **.prettierrc.json** file.
2. Add the following contents to it to ignore the transpiled source code:

```
dist/
```

The **node_modules/** folder is automatically ignored by Prettier.

Now that we have successfully set up Prettier, we are going to configure ESLint to enforce coding best practices.

Configuring ESLint

While Prettier focuses on the style and formatting of our code, ESLint focuses on the actual code, avoiding common mistakes or unnecessary code. Let's configure it now:

1. Delete the automatically created **.eslintrc.cjs** file.
2. Create a new **.eslintrc.json** file and start writing the following configuration into it. First, we set **root** to **true** to make sure ESLint does not look at parent folders for more configuration:

```
{  
  "root": true,
```

3. Define an **env** object, in which we set the browser environment to **true** so that ESLint understands browser-specific globals such as **document** and **window**:

```
  "env": {  
    "browser": true  
  },
```

4. Define a **parserOptions** object, where we specify that we are using the latest ECMAScript version and ESM:

```
  "parserOptions": {  
    "ecmaVersion": "latest",  
    "sourceType": "module"  
  },
```

5. Define an **extends** array to extend from recommended configurations. Specifically, we extend from ESLint's recommended rules and

the recommended rules for the plugins we installed:

```
"extends": [  
  "eslint:recommended",  
  "plugin:react/recommended",  
  "plugin:react/jsx-runtime",  
  "plugin:jsx-a11y/recommended",
```

6. As the last element of the array, we use **prettier** to disable all code style-related rules in ESLint and let Prettier handle it:

```
  "prettier"  
],
```

7. Now, we define settings for the plugins. First, we tell the **react** plugin to detect our installed React version automatically:

```
"settings": {  
  "react": {  
    "version": "detect"  
  }  
},
```

8. Finally, outside of the **settings** section, we define an **overrides** array, in which we specify that ESLint should only lint **.js** and **.jsx** files:

```
"overrides": [  
  {  
    "files": ["*.js", "*.jsx"]  
  }  
]
```

9. Create a new **.eslintignore** file, with the following contents:

```
dist/  
vite.config.js
```

The `node_modules/` folder is automatically ignored by ESLint.

10. Save the files and run `npx eslint src` in the Terminal to run the linter. You will see that there are some errors already due to our configured rules not matching the source provided by the default project in Vite:

```
❶ → ~/D/F/ch1 main: > npx eslint src
/Users/dan/Development/Full-Stack-React-Projects/ch1/src/App.jsx
12:9   error   Using target="_blank" without rel="noreferrer" (which implies rel="noopener") is a security risk in
older browsers: see https://mathiasbynens.github.io/rel-noopener/#recommendations  react/jsx-no-target-blank
15:9   error   Using target="_blank" without rel="noreferrer" (which implies rel="noopener") is a security risk in
older browsers: see https://mathiasbynens.github.io/rel-noopener/#recommendations  react/jsx-no-target-blank

* 2 problems (2 errors, 0 warnings)
2 errors and 0 warnings potentially fixable with the `--fix` option.
```

Figure 1.6 – When running ESLint for the first time, we get some errors about rule violations

11. Fortunately, all these issues are automatically fixable by ESLint. Run `npx eslint src --fix` to fix the issues automatically. Now, when you run `npx eslint src` again, you will not get any output. This means that there were no linter errors!

TIP

The `npx` command allows us to execute commands provided by `npm` packages, in a similar context as running them in `package.json` scripts would do. It can also run remote packages without installing them permanently. If the package is not installed yet, it will ask you whether it should do this.

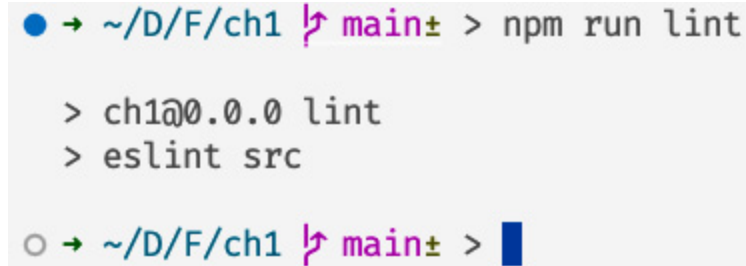
Adding a new script to run our linter

In the previous section, we have been calling the linter by running `npx eslint src` manually. We are now going to add a `lint` script to `package.json`:

1. In the Terminal, run the following command to define a `lint` script in the `package.json` file:

```
$ npm pkg set scripts.lint="eslint src"
```

2. Now, run `npm run lint` in the Terminal. This should execute `eslint src` successfully, just like `npx eslint src` did:



```
● → ~/D/F/ch1 ↗ main± > npm run lint  
  
  > ch1@0.0.0 lint  
  > eslint src  
  
○ → ~/D/F/ch1 ↗ main± > █
```

Figure 1.7 – The linter running successfully, with no errors

After setting up ESLint and Prettier, we still need to make sure that they run before we commit code. Let's set up Husky to make sure we commit proper code now.

Setting up Husky to make sure we commit proper code

After setting up Prettier and ESLint, we will now get our code automatically formatted on save by Prettier and see errors from ESLint in VS Code when we make mistakes or ignore best practices. However, we might miss some of these errors and accidentally commit code that is invalid. To avoid this, we can set up **Husky** and **lint-staged**, which run before we commit our code to Git and ensure that Prettier and ESLint are executed successfully on the source code before it is committed.

IMPORTANT

If you cloned the full repository for the book, Husky may not find the `.git` directory when running `npm install`. In that case, just run `git init` in the root of the corresponding chapter folder.

Let's set Husky and lint-staged up by following these steps:

1. Run the following command to install Husky and lint-staged as **dev** dependencies:

```
$ npm install --save-dev husky@8.0.3 \
  lint-staged@15.1.0
```

2. Open the **package.json** file and add the following **lint-staged** configuration to it in a new object after **devDependencies**, then save the file. This will run Prettier and ESLint on all committed **.js** and **.jsx** files and attempt to automatically fix code style and linter errors, if possible:

```
"lint-staged": {
  "**/*.{js,jsx}": [
    "npx prettier --write",
    "npx eslint --fix"
  ]
}
```

3. Initialize a Git repository in the **ch1** folder and make an initial commit with just the **package.json** file, as lint-staged does not get executed on the initial commit:

```
$ git init
$ git add package.json
$ git commit -m "chore: initial commit"
```

4. Add the **husky install** script to a **prepare** script in **package.json**, so that Husky gets installed automatically when the project is cloned and **npm install** is executed:

```
$ npm pkg set scripts.prepare="husky install"
```

5. Since we do not need to run **npm install** again right now, we need to manually run the **prepare** script this time:

```
$ npm run prepare
```

6. Add a **pre-commit** hook for lint-staged, so that ESLint and Prettier run every time we do **git commit**:

```
$ npx husky add .husky/pre-commit "npx lint-staged"
```

7. Now, add all files to Git and attempt to make a commit:

```
$ git add .  
$ git commit -m "chore: basic project setup"
```

If everything worked successfully, you should see **husky** running **lint-staged**, which, in turn, runs **prettier** and **eslint**, after you run **git commit**. If you are getting a configuration error, ensure that all files are saved properly and then run **git commit** again.

```
● → ~/D/F/ch1 ↵ main > git commit -m "chore: basic project setup"  
✓ Preparing lint-staged...  
✓ Running tasks for staged files...  
✓ Applying modifications from tasks...  
✓ Cleaning up temporary files...  
[main f226bcd] chore: basic project setup
```

Figure 1.8 – Husky and lint-staged successfully enforcing code style and best practices before we commit

Setting up commitlint to enforce a standard for our commit messages

In addition to linting our code, we can also lint our commit messages. You may have noticed that we were prefixing our commit messages with a type already (the **chore** type). Types make it easier to follow what was changed in a commit. To enforce the use of types, we can set up **commitlint**. Follow these steps to set it up:

1. Install commitlint and a conventional config for commitlint:

```
$ npm install --save-dev @commitlint/cli@18.4.3 \
  @commitlint/config-conventional@18.4.3
```

2. Create a new **.commitlintrc.json** file in the root of our project and add the following contents:

```
{
  "extends": ["@commitlint/config-conventional"]
}
```

3. Add a **commit-msg** hook to Husky:

```
$ npx husky add .husky/commit-msg \
  'npx commitlint --edit ${1}'
```

4. Now, if we try adding our changed files and committing without a type or a wrong type, we will get an error from commitlint and will not be able to make such a commit. If we add the correct type, it will succeed:

```
$ git add .
$ git commit -m "no type"
$ git commit -m "wrong: type"
$ git commit -m "chore: configure commitlint"
```

The following figure shows Husky in action. If we write an incorrect commit message, it will reject it and not let us commit the code. Only if we enter a properly formatted commit message will the commit go through:

```

➔ ~/D/F/ch1 | main: > git commit -m "no type"
➔ No staged files match any configured task.
✖ input: no type
✖ subject may not be empty [subject-empty]
✖ type may not be empty [type-empty]

✖ found 2 problems, 0 warnings
① Get help: https://github.com/conventional-changelog/commitlint/#what-is-commitlint

husky - commit-msg hook exited with code 1 (error)
➔ ~/D/F/ch1 | main: > git commit -m "wrong: type"
➔ No staged files match any configured task.
✖ input: wrong: type
✖ type must be one of [build, chore, ci, docs, feat, fix, perf, refactor, revert, style, test] [type-enum]

✖ found 1 problems, 0 warnings
① Get help: https://github.com/conventional-changelog/commitlint/#what-is-commitlint

husky - commit-msg hook exited with code 1 (error)
➔ ~/D/F/ch1 | main: > git commit -m "chore: configure commitlint"
➔ No staged files match any configured task.
[main e5e8cd0] chore: configure commitlint
4 files changed, 2019 insertions(+), 57 deletions(-)
create mode 100644 .commitlintrc.json
create mode 100755 .husky/commit-msg

```

Figure 1.9 – commitlint working successfully and preventing commits without a type and with wrong types

Commit messages in the commitlint conventional config

(<https://www.conventionalcommits.org/>) are structured in a way where a type must be listed first, then an optional scope follows, and then the description follows, such as **type(scope): description**.

Possible types are as follows:

- **fix:** For bug fixes
- **feat:** For new features
- **refactor:** For restructuring the code without adding features or fixing bugs
- **build:** For changes in the build system or dependencies
- **ci:** For changes in the CI/CD configuration
- **docs:** For changes in the documentation only
- **perf:** For performance optimizations
- **style:** For fixing code formatting
- **test:** For adding or adjusting tests

The scope is optional and best used in a monorepo to specify that changes were made to a certain app or library within it.

Summary

Now that we have successfully set up our project and started enforcing standards, we can continue working on our project without worrying about a consistent code style, consistent commit messages, or making small mistakes. ESLint, Prettier, Husky, and commitlint have got us covered.

In the next chapter, [***Chapter 2***](#), *Getting to Know Node.js and MongoDB*, we are going to learn how to write and run small Node.js scripts and how MongoDB, a database system, works.